

3.5 Hands-on Lab — Optimize and Serve a Model using Triton Inference Server

What you'll do

1. Export a pretrained **ResNet-50** to **ONNX**
 2. Build a **TensorRT** engine (FP16) from that ONNX
 3. Create a **Triton model repository** with `config.pbtxt`
 4. Run **Triton** in Docker and query it with the **Python client**
 5. Benchmark with **perf_analyzer** and enable **dynamic batching**
 6. (Optional) Compare ONNXRuntime vs TensorRT; try INT8
-

Prereqs (5–10 min)

- A GPU machine with working NVIDIA driver (`nvidia-smi` shows your GPU)
- Docker + NVIDIA Container Toolkit (so `docker run --gpus all ...` works)
- Outbound internet (to fetch containers + torchvision weights)
- ~5–10 GB free disk

Tip: If you completed Modules **1.5** and **2.5**, you're already set.

Part A — Pull the containers & set up workspace (2–3 min)

```
# 1) Choose Triton tag (adjust if you like)
export TRITON_TAG=24.06-py3

# 2) Create a workspace
mkdir -p ~/triton-lab/{export,models,resnet_test}
cd ~/triton-lab
```

Part B — Export ResNet-50 to ONNX (4–6 min)

We'll use the NVIDIA PyTorch container to export a pretrained ResNet-50 at input size 224×224.

```
# 1) Launch a PyTorch container with your workspace mounted
docker run --rm -it --gpus all \
  -v ~/triton-lab/export:/workspace \
  nvcr.io/nvidia/pytorch:24.06-py3 /bin/bash
```

Inside the container, run:

```
python - << 'PY'
import torch, torchvision
from pathlib import Path

# Get pretrained ResNet50 (torchvision downloads weights on first use)
model = torchvision.models.resnet50(weights="IMAGENET1K_V2").eval().cuda()

# Dummy input for export: NCHW = 1x3x224x224
dummy = torch.randn(1,3,224,224).cuda()

onnx_path = Path("/workspace/resnet50.onnx")
torch.onnx.export(
    model, dummy, onnx_path.as_posix(),
    input_names=["input"], output_names=["logits"],
    opset_version=17, do_constant_folding=True,
    dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}}
)
print("Exported:", onnx_path)
PY
exit
```

You should now have `~/triton-lab/export/resnet50.onnx`.

Part C — Build a TensorRT FP16 engine (3–6 min)

Use `trtexec` (included in Triton or TensorRT containers) to compile the ONNX into a `.plan` engine.

```
# Use the Triton image for convenience (trtexec is present)
docker run --rm -it --gpus all \
  -v ~/triton-lab/export:/workspace \
  nvcr.io/nvidia/tritonserver:${TRITON_TAG} \
  trtexec --onnx=/workspace/resnet50.onnx \
    --saveEngine=/workspace/resnet50_fp16.plan \
    --fp16 --workspace=4096 --minShapes=input:1x3x224x224 \
    --optShapes=input:8x3x224x224 --maxShapes=input:32x3x224x224
```

Result: `~/triton-lab/export/resnet50_fp16.plan`.

Note: `--min/opt/maxShapes` set your **optimization profiles** (dynamic shapes). Adjust based on typical batch sizes.

Part D — Create the Triton model repository (2–4 min)

We'll serve two variants to compare later: **TensorRT** and **ONNXRuntime**.

```
# 1) Make the repo layout
mkdir -p ~/triton-lab/models/resnet50_trt/1
mkdir -p ~/triton-lab/models/resnet50_onnx/1

# 2) Copy artifacts
cp ~/triton-lab/export/resnet50_fp16.plan ~/triton-lab/models/resnet50_trt/1/model.plan
cp ~/triton-lab/export/resnet50.onnx ~/triton-lab/models/resnet50_onnx/1/model.onnx
```

Create ~/triton-lab/models/resnet50_trt/config.pbtxt:

```
name: "resnet50_trt"
platform: "tensorrt_plan"
max_batch_size: 32
input [
  { name: "input"  data_type: TYPE_FP32 dims: [3, 224, 224] }
]
output [
  { name: "logits" data_type: TYPE_FP32 dims: [1000] }
]
instance_group [
  { kind: KIND_GPU, count: 1 }
]
dynamic_batching {
  preferred_batch_size: [ 8, 16, 32 ]
  max_queue_delay_microseconds: 2000
}
optimization { execution_accelerators { gpu_execution_accelerator : [ { name: "tensorrt" } ] } }
```

Create ~/triton-lab/models/resnet50_onnx/config.pbtxt:

```
name: "resnet50_onnx"
platform: "onnxruntime_onnx"
max_batch_size: 32
input [
  { name: "input"  data_type: TYPE_FP32 dims: [3, 224, 224] }
]
output [
  { name: "logits" data_type: TYPE_FP32 dims: [1000] }
]
instance_group [
  { kind: KIND_GPU, count: 1 }
]
dynamic_batching {
  preferred_batch_size: [ 8, 16, 32 ]
  max_queue_delay_microseconds: 2000
}
```

Why FP32 inputs? Even with an FP16 engine, TensorRT will happily cast FP32 inputs internally—this keeps client code simple.

Part E — Run Triton Server (1–2 min)

```
docker run --rm -it --gpus all --net=host \
  -v ~/triton-lab/models:/models \
  nvcr.io/nvidia/tritonserver:${TRITON_TAG} \
  tritonserver --model-repository=/models
```

Wait for “**READY**” logs. Triton will load both models.

Part F — Send a test request with the Python client (4–6 min)

Open a second terminal for the client container:

```
docker run --rm -it --net=host \
  nvcr.io/nvidia/tritonserver:${TRITON_TAG}-py3-sdk /bin/bash
```

Inside the client container:

```
pip install --no-cache-dir pillow numpy
```

```
python - << 'PY'
import numpy as np, requests
from PIL import Image

# 1) Load & preprocess an image to NCHW FP32 (224x224)
#   You can copy any JPEG/PNG onto your host and point to it via a bind mount,
#   or just fetch a local file path mounted into this container if you prefer.
img = Image.new("RGB", (256,256), (128,128,128)) # placeholder gray image
img = img.resize((224,224))
arr = np.array(img).astype(np.float32) / 255.0
# Normalize with ImageNet mean/std
mean = np.array([0.485, 0.456, 0.406], dtype=np.float32)
std  = np.array([0.229, 0.224, 0.225], dtype=np.float32)
arr = (arr/1.0 - mean) / std
# HWC -> CHW -> add batch
arr = np.transpose(arr, (2,0,1))[None, ...]

# 2) Build Triton HTTP/JSON request
def infer(model_name):
    url = f"http://localhost:8000/v2/models/{model_name}/infer"
    payload = {
        "inputs": [{
            "name": "input",
            "shape": list(arr.shape),
            "datatype": "FP32",
            "data": arr.flatten().tolist()
        }],
        "outputs": [{"name": "logits"}]
    }
    r = requests.post(url, json=payload, timeout=30)
    r.raise_for_status()
    out = np.array(r.json()["outputs"][0]["data"], dtype=np.float32).reshape((1,1000))
    print(model_name, "OK. Top-5 idx:", out.argsort(axis=1)[:,-5:])

infer("resnet50_onnx")
infer("resnet50_trt")
PY
```

You should see “OK” for both models with Top-5 indices printed (nonsense for the gray image, but proves end-to-end I/O).

Part G — Benchmark with perf_analyzer (3–6 min)

Still inside the **SDK** container:

```
# Try batch sizes and concurrency levels
perf_analyzer -m resnet50_trt -b 1  -u localhost:8000 -i http -p 2000 --concurrency-range 1:8
perf_analyzer -m resnet50_trt -b 8  -u localhost:8000 -i http -p 2000 --concurrency-range 1:8
perf_analyzer -m resnet50_trt -b 16 -u localhost:8000 -i http -p 2000 --concurrency-range 1:8

# Compare ONNXRuntime vs TensorRT
perf_analyzer -m resnet50_onnx -b 8  -u localhost:8000 -i http -p 2000 --concurrency-range 1:8
```

Look at **p50/p95 latency** and **inferences/sec**. You should see **TensorRT** beat ONNXRuntime, especially at higher batch/concurrency.

Part H – Tune dynamic batching & instances (2–5 min)

Stop the server (Ctrl-C) and tweak configs:

- Increase `preferred_batch_size` to `[ 8, 16, 32 ]` (already set)
- Raise `max_queue_delay_microseconds` to **4000–8000** if your SLA allows a few extra ms for bigger batches
- Add more instances:

```
instance_group [ { kind: KIND_GPU, count: 2 } ] # try 2 per GPU
```

Restart Triton and re-run `perf_analyzer` to see the impact.

(Optional) Part I – Quick INT8 path (advanced)

INT8 gives larger speedups but needs **calibration** images:

```
# Put ~500 representative RGB images into ~/triton-lab/export/calib/
# Then build INT8 engine with trtexec:
docker run --rm -it --gpus all \
  -v ~/triton-lab/export:/workspace \
  nvcr.io/nvidia/tritonserver:${TRITON_TAG} \
  trtexec --onnx=/workspace/resnet50.onnx \
    --saveEngine=/workspace/resnet50_int8.plan \
    --int8 --calib=/workspace/calib \
    --workspace=4096 \
    --minShapes=input:1x3x224x224 \
    --optShapes=input:8x3x224x224 \
    --maxShapes=input:32x3x224x224
```

Copy `resnet50_int8.plan` into a new model version (e.g., `models/resnet50_trt_int8/1/model.plan`) with a matching `config.pbtxt`, then benchmark. Expect higher throughput and lower latency (small accuracy trade-off).

(Optional) Part J – Compare ONNXRuntime vs TensorRT in one run

You already have both **resnet50_onnx** and **resnet50_trt** loaded—use `perf_analyzer` to gather head-to-head numbers and record them in a simple CSV for your course notes.

Troubleshooting quick hits

- **Triton won’t load model / shape errors** → Ensure ONNX input name (`input`) and dims `[3,224,224]` match `config.pbtxt`. Re-export with correct names if needed.
- **Engine fails at runtime** → Rebuild with shapes that cover your batch sizes; align `min/opt/maxShapes` with `max_batch_size`.

- **Poor throughput** → Increase `preferred_batch_size`, `instance_group count`, try FP16/INT8; verify GPU isn't memory-starved.
- **Client can't connect** → Ensure `--net=host` on both server and SDK containers; verify `localhost:8000` responds (`/v2/health/ready`).
- **OOM during build** → Reduce `--workspace`, lower `maxShapes`, or build on a larger GPU.

Clean up

```
# Stop server (Ctrl-C), then remove large images if desired
docker image prune -f
docker rmi nvcr.io/nvidia/tritonserver:${TRITON_TAG} \
    nvcr.io/nvidia/tritonserver:${TRITON_TAG}-py3-sdk \
    nvcr.io/nvidia/pytorch:24.06-py3 || true
```
